



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

**THIẾT BỊ ĐEO THEO DÕI SỨC KHỎE, PHÁT HIỆN TẾ
NGÃ VÀ ĐỊNH VỊ DÙNG ESP32-S3**

BÁO CÁO CÔNG VIỆC CÁ NHÂN

Người thực hiện	Hồ Sĩ Phú
Mã số sinh viên	24207101
Nhóm	03 thành viên
Phạm vi báo cáo	Đo nhịp tim và ước lượng SpO ₂ bằng MAX30102

Phạm Hùng Tiến – Hồ Sĩ Phú – Quách Gia Thịnh

Mã nguồn đồ án: github.com/PhamHungTien/esp32-s3-health-monitor

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

Nội dung phụ trách

Trong đồ án, em phụ trách cảm biến MAX30102: cấu hình thanh ghi, đọc FIFO, xử lý tín hiệu RED/IR, phát hiện ngón tay, tính nhịp tim và ước lượng SpO₂.

1. Mục tiêu công việc

Phần xử lý cần chuyển dữ liệu RED/IR thành BPM và SpO₂, đồng thời nhận biết trường hợp chưa đặt ngón tay hoặc tín hiệu chưa ổn định. Giá trị chỉ được hiển thị khi thỏa các điều kiện về biên độ và khoảng nhịp.

2. Công việc đã thực hiện

2.1. Cấu hình và thu FIFO

Em cấu hình MAX30102 bằng các thanh ghi điều khiển: reset, con trỏ FIFO, chế độ SpO₂, độ phân giải ADC, tốc độ lấy mẫu 100 mẫu/giây và dòng LED. Mỗi phần tử FIFO gồm ba byte RED và ba byte IR, được ghép thành số 18 bit. Số mẫu chờ được xác định từ hai con trỏ ghi/đọc để tránh mất dữ liệu.

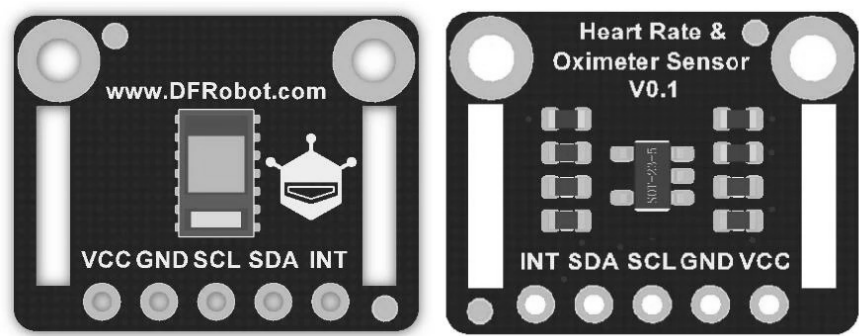


Figure 1: Mô-đun tham chiếu dùng MAX30102 với các chân VCC, GND, SCL, SDA và INT. Bo thực tế của nhóm được xác nhận bằng địa chỉ I2C 0x57; hình dạng PCB có thể khác. Nguồn ảnh: DFRobot.

Thanh ghi	Giá trị	Chức năng trong chương trình
0x09	0x40/0x03	Reset, sau đó chạy chế độ RED–IR
0x04–0x06	0x00	Xóa con trỏ ghi, tràn và đọc FIFO
0x08	0x1F	Cho phép FIFO quay vòng, không lấy trung bình trong chip
0x0A	0x27	Thang ADC 4.096 nA, 100 mẫu/giây, xung 411 μ s
0x0C/0x0D	0x24	Dòng LED đỏ và hồng ngoại khoảng 7,2 mA

Mỗi mẫu được gán thời điểm tăng 10 ms. Thời gian mẫu do đó không phụ thuộc vào thời gian vẽ OLED hay in Serial, giúp khoảng RR được tính nhất quán.

2.2. Phát hiện ngón tay và tiền xử lý

Giá trị IR nền khi không đặt tay đo được khoảng 330–400, trong khi đặt tay đạt khoảng 132.000–137.000. Em dùng hai ngưỡng nhận/nhả để trạng thái ngón tay không thay đổi liên tục. Khi rời tay, bộ đếm nhịp, cửa sổ AC/DC và các giá trị đo được xóa.

Thành phần DC được ước lượng bằng bộ lọc IIR chậm. Thành phần AC là hiệu giữa mẫu và DC; nó tiếp tục được làm mượt để giảm nhiễu cao tần. Tín hiệu sử dụng số thực đơn và không cần thư viện xử lý tín hiệu bên ngoài.

```

1 // Chọn ngưỡng IR theo trạng thái trước đó để tạo vùng trễ khi nhận hoặc bỏ ngón tay.
2 bool hasFinger = ppg.fingerPresent ?
3
4 // Khi đã nhận ngón tay dùng ngưỡng 7.000; khi chưa nhận dùng ngưỡng 10.000 để chống dao động.
5 (irValue > 7000) : (irValue > 10000);
6
7 // Đi vào nhánh xử lý khi mẫu hiện tại không còn đủ điều kiện có ngón tay.
8 if (!hasFinger) {
9
10 // Nếu vừa chuyển từ có sang không có ngón tay thì xóa các kết quả đo của phiên trước.
11 if (ppg.fingerPresent) PPG_ResetMetrics();
12
13 // Đặt mức nền kênh đỏ bằng mẫu hiện tại để bộ lọc khởi động lại đúng mức tín hiệu.
14 state.dcRed = redValue;
15
16 // Đặt mức nền kênh hồng ngoại theo cùng nguyên tắc.
17 state.dcIr = irValue;
18
19 // Dừng xử lý mẫu vì không được phép tính BPM hoặc SpO2 khi chưa có ngón tay.
20 return;
21
22 // Kết thúc nhánh không có ngón tay.
23 }
24
25 // Dòng trống tách phần loại bỏ mẫu với phần khởi tạo một phiên đo mới.
26
27 // Kiểm tra đây có phải mẫu hợp lệ đầu tiên sau khi đặt ngón tay hay không.
28 if (!ppg.fingerPresent) {
29
30 // Ghi nhận cảm biến đang có ngón tay.
31 ppg.fingerPresent = true;
32
33 // Lưu thời điểm bắt đầu phiên đo để các bước sau biết tuổi của tín hiệu.
34 state.fingerStartMs = sampleTimeMs;
35
36 // Khởi tạo mức nền kênh đỏ bằng mẫu đầu tiên của phiên đo.
37 state.dcRed = redValue;
38
39 // Khởi tạo mức nền kênh hồng ngoại bằng mẫu đầu tiên của phiên đo.
40 state.dcIr = irValue;
41
42 // Kết thúc khối khởi tạo phiên đo.
43 }

```

Đoạn mã 1: Ngưỡng trễ nhận ngón tay và xóa trạng thái khi rời tay

2.3. Thuật toán nhịp tim

Với cảm biến đặt ở đầu ngón, tín hiệu IR sau lọc dao động quanh đường nền. Trong 0,8 giây đầu, bộ lọc bám nhanh theo mức DC để loại xung lớn khi vừa đặt tay. Sau đó, đây cục bộ vượt ngưỡng thích nghi được dùng làm mốc nhịp; khoảng trơ ngắn một nhịp bị đếm hai lần. Khoảng giữa hai

nhịp hợp lệ Δt được đổi sang BPM:

$$\text{BPM} = \frac{60\,000}{\Delta t_{\text{ms}}}.$$

Chỉ các giá trị trong miền sinh lý được đưa vào bộ làm mượt. Chương trình bỏ 0,8 giây đầu để đường nền ổn định; khoảng RR hợp lệ đầu tiên sau đó được hiển thị ngay. Các nhịp kế tiếp chỉ được dùng nếu không lệch quá 22 BPM so với giá trị đang có. Trong điều kiện đặt tay tốt, kết quả thường xuất hiện sau khoảng 2–3 giây.

2.4. Ước lượng SpO_2

Trong mỗi cửa sổ, em tính thành phần DC và độ mạnh AC của hai kênh. Tỷ số chuẩn hóa được xác định bởi:

$$R = \frac{AC_{\text{RED}}/DC_{\text{RED}}}{AC_{\text{IR}}/DC_{\text{IR}}}, \quad \text{SpO}_2 \approx a - bR.$$

Kết quả được chặn trong miền hợp lệ, làm mượt giữa các cửa sổ và chỉ công nhận khi biên độ AC đủ lớn, DC khác 0 và tỷ số nằm trong vùng hợp lý. Chỉ số chất lượng Q được giữ trong nhật ký Serial để hỗ trợ kiểm tra; màn hình người dùng không hiện nhãn TỐT/VỪA/YẾU.

Giới hạn đo SpO_2

Giá trị SpO_2 hiện là ước lượng phục vụ đồ án, chưa phải thiết bị y tế. Các lần thử gần nhất cho kết quả khoảng 90–96%; cần thu thập dữ liệu với máy đo chuẩn, khớp lại hệ số a, b và kiểm tra trên nhiều người trước khi kết luận độ chính xác.

3. Các đoạn mã quyết định

3.1. Xác nhận nhịp và khởi tạo BPM

Bộ dò sử dụng đầy cục bộ, ngưỡng theo biên độ tín hiệu và khoảng trễ 450 ms. Khoảng RR hợp lệ đầu tiên được xuất ngay; các nhịp sau chỉ được nhận nếu không lệch quá 22 BPM so với giá trị đang có.

```

1 // Tính độ dốc của tín hiệu đã lọc so với mẫu ngay trước đó.
2 float slope = state.filtered - state.previousFiltered;
3
4 // Đặt ngưỡng đỉnh bằng 95% biên độ bao hiện tại để ngưỡng tự thích nghi với cường độ tín hiệu.
5 float peakThreshold = state.envelope * 0.95f;
6
7 // Giới hạn ngưỡng tối thiểu ở 100 để nhiễu nhỏ không bị nhận nhầm thành nhịp.
8 if (peakThreshold < 100.0f) peakThreshold = 100.0f;
9
10 // Dòng trống phân tách bước chuẩn bị ngưỡng với bước xác nhận đỉnh.
11
12 // Phát hiện thời điểm độ dốc đổi từ âm sang không âm, tức một cực tiểu của dạng sóng đã đảo.
13 if (state.previousSlope < 0.0f && slope >= 0.0f &&
14     // Yêu cầu biên độ cực tiểu vượt ngưỡng thích nghi vừa tính.
15     state.previousFiltered < -peakThreshold &&
16     // Bước hai nhịp cách nhau hơn 450 ms để loại bỏ đỉnh giả quá gần.
17     now - ppg.lastBeatMs > 450) {
18     // Chỉ tính khoảng RR khi hệ thống đã lưu được thời điểm của một nhịp trước.
19     if (ppg.lastBeatMs != 0) {
20         // Tính khoảng thời gian giữa hai nhịp liên tiếp theo mili giây.
21         uint32_t interval = now - ppg.lastBeatMs;
22
23         // Chỉ chấp nhận khoảng RR từ 480 đến 1.500 ms, tương ứng vùng nhịp hợp lý của nguyên mẫu.
24         if (interval >= 480 && interval <= 1500) {
25             // Đổi khoảng RR sang nhịp mỗi phút bằng công thức 60.000 chia cho số mili giây.
26             float instantBPM = 60000.0f / interval;
27
28             // Kiểm tra đây có phải giá trị BPM hợp lệ đầu tiên của phiên đo hay không.
29             if (!ppg.heartRateValid) {
30                 // Hiển thị trực tiếp BPM tức thời ở nhịp hợp lệ đầu tiên.
31                 ppg.bpm = instantBPM;
32
33                 // Đánh dấu BPM đã sẵn sàng để giao diện không tiếp tục hiện dấu chờ.
34                 ppg.heartRateValid = true;
35
36                 // Với các nhịp sau, chỉ nhận giá trị mới nếu chênh lệch không quá 22 BPM.
37                 } else if (fabsf(instantBPM - ppg.bpm) <= 22.0f) {
38                     // Làm mượt bằng 80% giá trị cũ và 20% giá trị mới để giảm rung số.
39                     ppg.bpm = 0.80f * ppg.bpm + 0.20f * instantBPM;
40
41                     // Kết thúc nhánh cập nhật BPM sau khi đã có kết quả trước đó.
42                 }
43
44                 // Kết thúc khối kiểm tra khoảng RR hợp lệ.
45             }
46
47             // Kết thúc khối yêu cầu phải có nhịp trước.
48         }
49
50         // Lưu thời điểm đỉnh hiện tại; đỉnh này sẽ làm mốc cho lần tính RR tiếp theo.
51         ppg.lastBeatMs = now;

```

```

41 // Kết thúc điều kiện xác nhận định nhịp.
42 }

```

Đoạn mã 2: Điều kiện phát hiện nhịp và hiển thị BPM sớm

Đây là nhánh quyết định trực tiếp khi nào BPM được phép xuất hiện trên màn hình và cách giá trị tiếp tục được làm mượt.

3.2. Ratio-of-ratios cho SpO₂

```

1 // Tính giá trị hiệu dụng của thành phần AC kênh đỏ từ tổng bình phương trong cửa sổ.
2 float redRms = sqrtf((float)(state.redSquareSum /
3 // Chia tổng bình phương cho số mẫu trước khi lấy căn để nhận RMS đúng chuẩn.
4 state.windowSamples));
5 // Bắt đầu phép tính RMS tương tự cho kênh hồng ngoại.
6 float irRms = sqrtf((float)(state.irSquareSum /
7 // Hoàn tất phép tính RMS kênh hồng ngoại theo số mẫu của cửa sổ.
8 state.windowSamples));
9 // Tính tỷ số chuẩn hóa giữa AC/DC của kênh đỏ và AC/DC của kênh hồng ngoại.
10 float ratio = (redRms / state.dcRed) / (irRms / state.dcIr);
11 // Dòng trống tách phép tính tỷ số với bước kiểm tra miền hợp lệ.
12
13 // Chỉ ước lượng SpO2 khi tỷ số nằm trong miền 0,2 đến 2,0 để loại mẫu bất thường.
14 if (ratio > 0.2f && ratio < 2.0f) {
15 // Áp dụng đường hiệu chuẩn tuyến tính của nguyên mẫu để đổi tỷ số quang sang phần trăm SpO2.
16 float estimatedSpO2 = constrain(110.0f - 25.0f * ratio,
17 // Giới hạn kết quả trong khoảng 70–100% để tránh giá trị hiển thị phi thực tế.
18 70.0f, 100.0f);
19 // Kiểm tra hệ thống đã có kết quả SpO2 hợp lệ từ cửa sổ trước hay chưa.
20 ppg.spo2 = ppg.spo2Valid ?
21 // Nếu đã có thì làm mượt 80/20; nếu chưa có thì dùng ngay ước lượng đầu tiên.
22 0.8f * ppg.spo2 + 0.2f * estimatedSpO2 : estimatedSpO2;
23 // Kết thúc nhánh cập nhật SpO2.
24 }

```

Đoạn mã 3: Tính tỷ số quang và giá trị SpO2 nguyên mẫu

Hai hệ số 110 và 25 là đường cong nguyên mẫu. Báo cáo không xem chúng là hệ số y tế đã hiệu chuẩn.

4. Nhật ký khối lượng

Hạng mục	Thời lượng	Đầu ra
Cấu hình thanh ghi và FIFO 100 Hz	5 giờ	Luồng RED/IR liên tục
Phát hiện ngón tay và reset trạng thái	4 giờ	Không xuất số khi rời tay
Lọc DC/AC và ngưỡng thích nghi	5 giờ	Tín hiệu xung ổn định
Phát hiện nhịp, khoảng trơ, BPM	7 giờ	BPM quan sát 72–83
Ratio-of-ratios và làm mượt SpO ₂	6 giờ	Giá trị cùng cỡ hợp lệ
Đánh giá chất lượng, thử nghiệm	3 giờ	Q, bảng kết quả và giới hạn
Tổng	30 giờ	Đã cài đặt; SpO₂ cần hiệu chuẩn

5. Kết quả kiểm thử

Tình huống	Quan sát	Kết luận
Không đặt ngón tay	IR khoảng 330–400; BPM/SpO ₂ ở WAIT	Đạt
Đặt ngón tay ổn định	IR khoảng 132.000–137.000	Nhận diện biên độ rõ
Sau thời gian hội tụ	BPM khoảng 72–83 qua các lần thử	Đạt
Ước lượng SpO ₂ thử nghiệm	Khoảng 90–96% trong các lần thử gần nhất	Cần hiệu chuẩn
Rút ngón tay	Xóa cỡ hợp lệ và cửa sổ đo	Đạt

6. Kết quả đạt được

Các giá trị đầu ra

Phần chương trình MAX30102 gồm các hàm khởi tạo, đọc FIFO, xử lý từng mẫu, phát hiện ngón tay, tính BPM, tính ratio-of-ratios và đánh giá chất lượng tín hiệu. Các giá trị dùng chung gồm `fingerPresent`, `bpm`, `spo2`, `signalQuality`, `heartRateValid` và `spo2Valid`.

7. Nhận xét và hướng hoàn thiện

Thuật toán đã phát hiện được ngón tay và cho BPM ổn định trong điều kiện người đo giữ yên. SpO_2 hiện mới là giá trị ước lượng. Cần thu thêm dữ liệu đối chứng để hiệu chỉnh hệ số và đánh giá ảnh hưởng của lực ép, chuyển động và ánh sáng ngoài.